(DEEMED TO BE UNIVERSITY)

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally,Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	MD Razia
Student Name	P Tanvitha
Roll Number	2520030385
Branch	CSE
Course Outcome	CO1

Case Study Topic: MediFlow Hospital – Appointment-Driven Patient Indexing System using BST and AVL Trees

Work Done Summary:

Implemented Binary Search Tree (BST) and AVL Tree data structures in Java keyed by Patient ID for managing hospital patient records. Inserted patient records according to appointment-order IDs provided by the hospital registration system. Constructed a plain BST and analyzed its height and lookup performance against the hospital's SLA requirements. Implemented AVL Tree balancing operations to maintain efficient search performance even with nearly sorted insertion sequences. Performed patient record insertion, search, and deletion operations for transferred, discharged, and admission-closed patients. Compared the performance of BST and AVL Trees and demonstrated that AVL Trees provide guaranteed O(log n) search, insertion, and deletion times, making them suitable for real-time patient indexing and triage systems.

Implementation Details:

1. Created BST/AVL Node class with fields: patientId, left, right, and height (for AVL).
2. `insert()` recursively inserts patient records into the BST/AVL Tree based on appointment-order Patient IDs.
3. `search()` quickly locates patient records for triage and emergency lookup operations.
4. `delete()` removes patient records of transferred, discharged, or admission-closed patients.
5. `inorderTraversal()` displays patient IDs in sorted order for administrative reporting.
6. `calculateHeight()` determines the height of the BST and evaluates worst-case lookup performance.
7. `rotateLeft()` and `rotateRight()` perform AVL balancing operations after insertions and deletions.
8. `balanceFactor()` checks node balance and triggers LL, RR, LR, or RL rotations when necessary.
9. Performance Analysis compares BST height and AVL height against the hospital's lookup SLA.
10. Main Method demonstrates patient record insertion, searching, deletion, traversal, AVL balancing, and performance comparison.

Result: Screenshots / Output

```
=== MEDIFLOW HOSPITAL PATIENT INDEXING ===

Patient IDs Inserted:
20 30 35 40 45 50 60 65 70 75 80 85 90
BST Inorder Traversal:
20 30 35 40 45 50 60 65 70 75 80 85 90
BST Height (edges): 12
Search 60 in BST: Found
Search 55 in BST: Not Found
BST Inorder After Deletions: 20 35 40 45 60 65 75 80 85 90
------------------------------------------------------------
AVL Inorder Traversal:
20 30 35 40 45 50 60 65 70 75 80 85 90
AVL Height (edges): 3
Search 60 in AVL: Found
Search 55 in AVL: Not Found
AVL Inorder After Deletions: 20 35 40 45 60 65 75 80 85 90
------------------------------------------------------------
Performance Comparison:
BST Height   = 12
AVL Height   = 3

Observation: AVL Tree maintains balanced structure and provides O(log n) performance.
```

The AVL Tree maintained a balanced structure and achieved a height of 3, whereas the plain BST reached a height of 12 due to nearly sorted insertions. Hence, AVL Trees satisfy the hospital's fast patient lookup requirements more effectively than plain BSTs.

GitHub Link: https://github.com/pondurutanvitha-stack/DSA-2

Student Signature	Faculty Signature